

Linear Regression — Errors, Formulas, Gradients & Raw NumPy Implementation

1. Regression: Basic Idea

Linear regression predicts a **continuous numerical value**.

For a single feature:

$$\hat{y} = mx + b$$

For multiple features:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_px_p + b$$

where:

- $\hat{y}$ = predicted value
- y = actual value
- m = slope/weight in simple linear regression
- w_j = weight of feature j
- b = intercept/bias
- p = number of features

The prediction error for observation i is:

$$e_i = \hat{y}_i - y_i$$

2. Regression Errors / Evaluation Metrics

MAE — Mean Absolute Error

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Meaning

Average absolute difference between actual and predicted values.

Example:

```
Actual      = 10
Predicted   = 7
Error       = |10 - 7| = 3
```

Advantages

- Easy to understand.
- Same units as the target.
- Less sensitive to outliers than MSE.

Disadvantage

The absolute value function is not differentiable at 0, so it is less convenient as the basic loss for gradient descent.

MSE — Mean Squared Error

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Meaning

Average of squared prediction errors.

Because the error is squared, large errors are penalized heavily.

```
error = 2   →   4
error = 10  →  100
```

Important

For our gradient-descent implementation, **MSE is the main loss function**.

RMSE — Root Mean Squared Error

$$RMSE = \sqrt{MSE}$$

or:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

RMSE has the **same units as the target variable**.

For example, if house price is measured in ₹:

```
MSE → ₹2
RMSE → ₹
```

RMSE is also more sensitive to large errors/outliers.

R^2 — R-Squared

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

where:

$$SS_{res} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

and:

$$SS_{tot} = \sum_{i=1}^n (y_i - \bar{y})^2$$

So:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Intuition

R^2 compares your model against a baseline that always predicts the mean $\bar{y}$.

```
R2 = 1    → perfect fit
R2 = 0    → equivalent to mean prediction
R2 < 0    → worse than mean prediction
```

Important: R^2 is not "prediction accuracy percentage."

MAPE — Mean Absolute Percentage Error

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

It expresses error as a percentage.

Problem

MAPE becomes problematic when actual values are zero or very close to zero.

Quick Comparison

Metric	Formula	Outlier Sensitivity	Same Units as y ?		
MAE	$\frac{1}{n} \sum$	$y - \hat{y}$	\$	Low	Yes
MSE	$\frac{1}{n} \sum (y - \hat{y})^2$	High	No		
RMSE	$\sqrt{MSE}$	High	Yes		
R^2	$1 - \frac{SS_{res}}{SS_{tot}}$	Depends	No		
MAPE	$\frac{100}{n} \sum$	$\frac{y - \hat{y}}{y}$	\$	High/problematic near 0	Percentage

Remember

***MSE** → commonly used as the loss for gradient descent.*

***MAE** → average absolute error.*

RMSE → error in the original target units.

R^2 → how much variation the model explains compared with the mean baseline.

3. Simple Linear Regression

With one feature:

$$\hat{y} = mx + b$$

The MSE loss is:

$$J(m, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Substituting $\hat{y}_i = mx_i + b$:

$$J(m, b) = \frac{1}{n} \sum_{i=1}^n (mx_i + b - y_i)^2$$

Gradient with respect to m

$$\frac{\partial J}{\partial m} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_i$$

Gradient with respect to b

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

Gradient descent updates:

$$m \leftarrow m - \alpha \frac{\partial J}{\partial m}$$

$$b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

where α is the learning rate.

4. Simple Linear Regression — Raw NumPy

No `sklearn` — only NumPy.

```
import numpy as np

# Data
x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

# Parameters
m = 0
b = 0

lr = 0.001
epochs = 1000

n = x.size

# Gradient Descent
for i in range(epochs):

    # Prediction
    y_pred = m * x + b

    # Error
    error = y_pred - y

    # Gradients
    dm = 2 * np.sum(error * x) / n
    db = 2 * np.sum(error) / n

    # Parameter update
    m = m - lr * dm
    b = b - lr * db

print("m:", m)
print("b:", b)

# Final predictions
y_pred = m * x + b

print("Predictions:", y_pred)
```

For this dataset, the learned values should approach:

```
m ≈ 2  
b ≈ 0
```

Therefore:

```
x = 1 → ŷ ≈ 2  
x = 2 → ŷ ≈ 4  
x = 3 → ŷ ≈ 6  
...
```

5. Multiple Linear Regression

Now suppose we have multiple features.

The model is:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_px_p + b$$

Instead of storing each weight separately, we use a vector:

$$w = \begin{bmatrix} w_1 & w_2 & \vdots & w_p \end{bmatrix}$$

Then the model becomes:

$$\hat{y} = Xw + b$$

Here:

- X = feature matrix
- w = weight vector
- b = bias

For example:

```
X =  
[  
    [x11, x12],  
    [x21, x22],  
    [x31, x32]  
]  
  
w =  
[  
    w1,  
    w2  
]
```

Prediction:

```
y_pred = X @ w + b
```

6. Multiple Linear Regression — Gradients

Define:

$$e = \hat{y} - y$$

MSE:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

The gradient for **all weights** is:

$$\frac{\partial J}{\partial w} = \frac{2}{n} X^T e$$

The gradient for the bias is:

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum_{i=1}^n e_i$$

In NumPy:


```
dw = 2 * (X.T @ error) / n
db = 2 * np.sum(error) / n
```

And the updates are:

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w}$$

$$b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

7. Why `X.T @ error`?

Suppose:

```
X.shape = (5, 2)
```

That means:

```
5 observations
2 features
```

So:

```
X.T.shape = (2, 5)
error.shape = (5,)
```

Therefore:

```
X.T @ error
```

produces:

```
(2, 5) @ (5,)
      ↓
      (2,)
```

The result contains:

```
[  
    gradient_for_w1,  
    gradient_for_w2  
]
```

It is equivalent to:

```
np.dot(X.T, error)
```

in this case.

8. Multiple Linear Regression — Raw NumPy

```
import numpy as np

# -----
# Data
# -----
X = np.array([
    [1, 2],
    [2, 3],
    [3, 4],
    [4, 5],
    [5, 6]
])
y = np.array([5, 8, 11, 14, 17])
# -----
# Parameters
# -----
n = X.shape[0]
num_features = X.shape[1]
w = np.zeros(num_features)
b = 0
lr = 0.001
epochs = 1000
# -----
# Gradient Descent
# -----

for i in range(epochs):

    # Prediction
    y_pred = X @ w + b

    # Error
    error = y_pred - y

    # Gradients
    dw = 2 * (X.T @ error) / n
    db = 2 * np.sum(error) / n

    # Update
    w = w - lr * dw
    b = b - lr * db

# -----
# Results
# -----
print("Weights:", w)
print("Bias:", b)
# Predictions
```

```
y_pred = X @ w + b
print("Predictions:", y_pred)
```

9. The Core Pattern to Memorize

Simple Linear Regression

```
y_pred = m * x + b

error = y_pred - y

dm = 2 * np.sum(error * x) / n
db = 2 * np.sum(error) / n

m -= lr * dm
b -= lr * db
```

Mathematically:

$$\hat{y} = mx + b$$

$$\frac{\partial J}{\partial m} = \frac{2}{n} \sum ex$$

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum e$$

Multiple Linear Regression

```
y_pred = X @ w + b

error = y_pred - y

dw = 2 * (X.T @ error) / n
db = 2 * np.sum(error) / n

w -= lr * dw
b -= lr * db
```

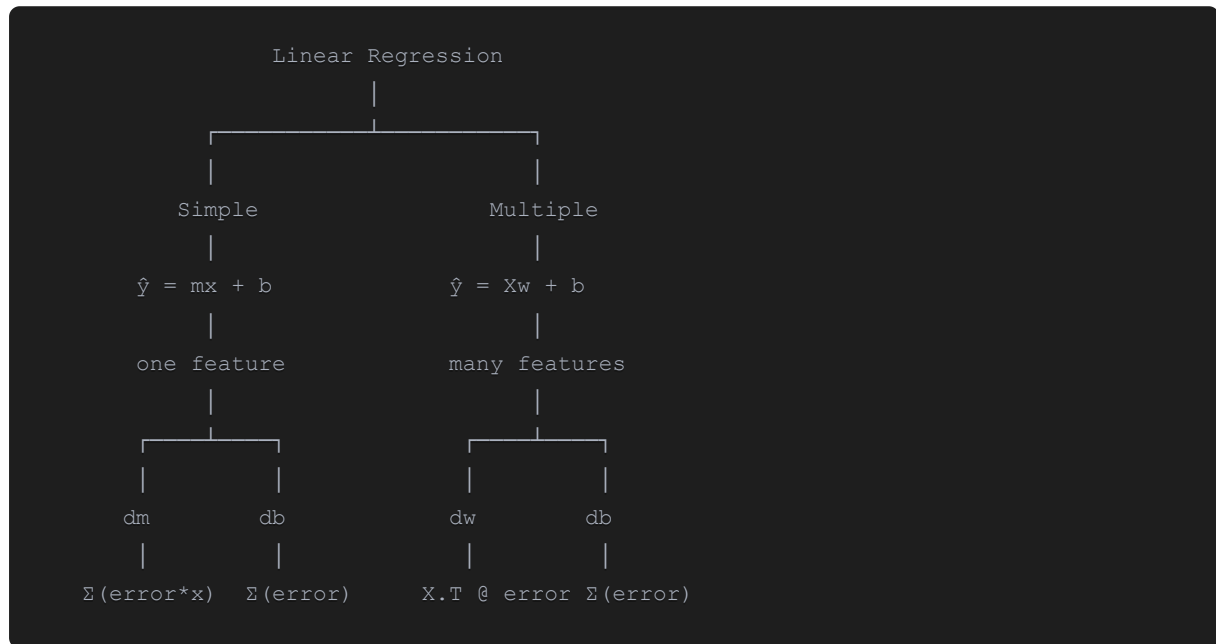
Mathematically:

$$\hat{y} = Xw + b$$

$$\frac{\partial J}{\partial w} = \frac{2}{n} X^T e$$

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum e$$

10. The Big Picture



The fundamental workflow is always:

1. Initialize parameters
- ↓
2. Make predictions
- ↓
3. Calculate error
- ↓
4. Calculate gradients
- ↓
5. Update parameters
- ↓
6. Repeat

The most important formulas to remember are:

$$\hat{y} = mx + b$$

$$\hat{y} = Xw + b$$

$$\frac{\partial J}{\partial m} = \frac{2}{n} \sum ex$$

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum e$$

$$\frac{\partial J}{\partial w} = \frac{2}{n} X^T e$$